

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
```

```
x=np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y_reg=np.array([30,40,45,55,65,70,60,90])
y_cls=np.array([0,0,0,0,1,1,1,1])
```

```
lin=LinearRegression()
lin.fit(x,y_reg)
print('Regression output:',lin.predict([[0.73]]))
```

Regression output: [29.72202381]

```
log=LogisticRegression()
log.fit(x,y_cls)
print('Classification output:',log.predict([[0.73]]))
```

```
print('Probability output:',log.predict_proba([[0.73]]))
```

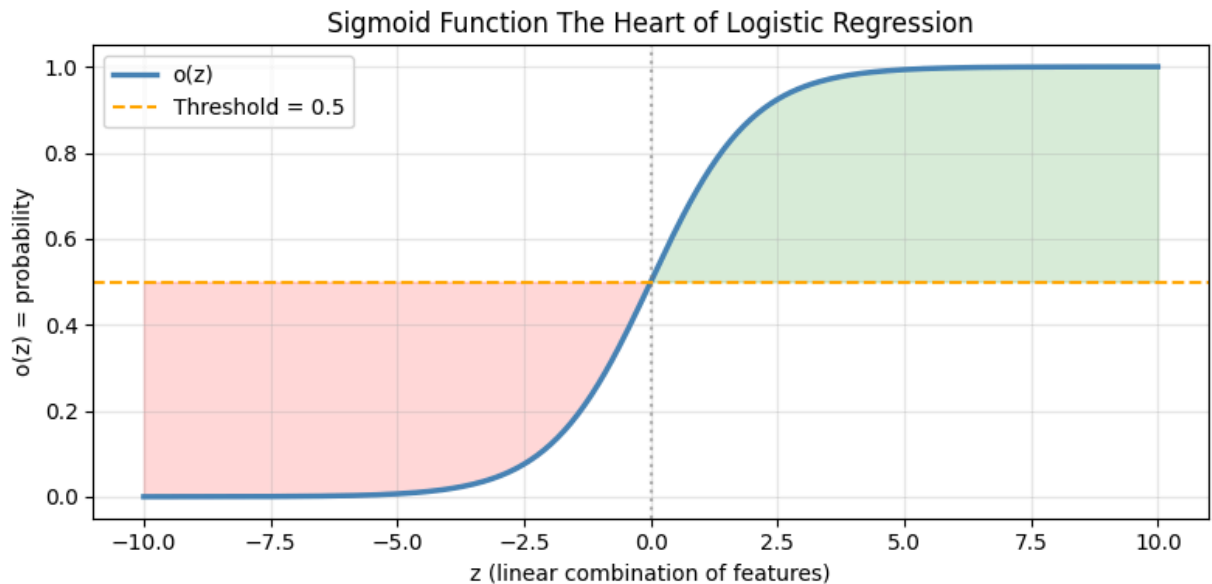
Classification output: [0]
Probability output: [[0.98799263 0.01200737]]

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def sigmoid(z):
    return 1/(1+np.exp(-z))
```

```
z=np.linspace(-10,10,300)
```

```
prob = sigmoid(z)
plt.figure(figsize=(8,4))
plt.plot(z, prob, color='steelblue', linewidth=2.5, label='o(z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)
plt.fill_between(z, prob, 0.5, where=(prob > 0.5), alpha=0.15, color='green')
plt.fill_between(z, prob, 0.5, where=(prob < 0.5), alpha=0.15, color='red')
plt.xlabel('z (linear combination of features)')
plt.ylabel('o(z) = probability')
plt.title('Sigmoid Function The Heart of Logistic Regression')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
print(f'sigmoid(0) = {sigmoid(0):.4f}')
print(f'sigmoid(5) = {sigmoid(5):.4f}')
print(f'sigmoid(-5) = {sigmoid(-5):.4f}')
```



```
sigmoid(0) = 0.5000
sigmoid(5) = 0.9933
sigmoid(-5) = 0.0067
```

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split
```

```
x,y=make_classification(n_samples=500, n_features=5,weights=[0.7,0.3],random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
model=LogisticRegression();
model.fit(x_train,y_train)
probs=model.predict_proba(x_test)[:,:1]
```

```
print(f'{'Threshold':>10} {'Precision':>10} {'Recall':>8} {'F1':>6}')
print('-'*45)
for threshold in [0.2,0.3,0.4,0.5,0.6,0.7,0.8]:
    preds=(probs>=threshold).astype(int)
    p=precision_score(y_test,preds,zero_division=0)
    r=recall_score(y_test,preds)
    f=f1_score(y_test,preds,zero_division=0)
    print(f'{'threshold':>10.1f} {'p':>10.3f} {'r':>8.3f} {'f':>6.3f}')
```

Threshold	Precision	Recall	F1
0.2	0.787	0.913	0.846
0.3	0.831	0.855	0.843
0.4	0.905	0.826	0.864
0.5	0.898	0.768	0.828
0.6	0.891	0.710	0.790
0.7	0.918	0.652	0.763
0.8	0.972	0.507	0.667

```
import numpy as np
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
y_true=[1,1,1,1,1, 0,0,0,0,0, 1,0,1,0,1]
y_pred=[1,1,1,0,0, 0,0,0,1,1, 1,0,0,1,1]
```

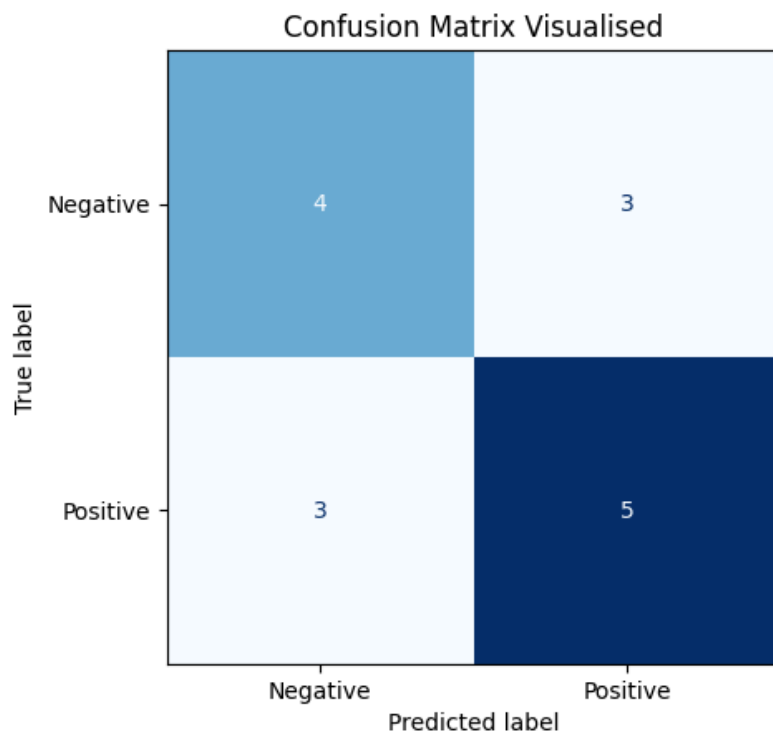
```
y_true=np.array(y_true)
y_pred=np.array(y_pred)
```

```
TP=((y_pred==1) & (y_true==1)).sum()
TN=((y_pred==0) & (y_true==0)).sum()
FP=((y_pred==1) & (y_true==0)).sum()
FN=((y_pred==0) & (y_true==1)).sum()
```

```
print(f'TP = {TP}, TN={TN}, FP={FP}, FN={FN}')
print(f'Accuracy = (TP+TN)/Total = {(TP+TN)/len(y_true)} = {(TP+TN)/len(y_true):.2f}')
print(f'Precision = TP/(TP+FP) = {TP}/{(TP)+{FP}} = {TP/(TP+FP):.2f}')
print(f'Recall = TP/(TP+FN) = {TP}/{(TP)+{FN}} = {TP/(TP+FN):.2f}')

cm=confusion_matrix(y_true,y_pred)
print('Confusion matrix:\n', cm)
disp=ConfusionMatrixDisplay(cm,display_labels=['Negative','Positive'])
disp.plot(cmap='Blues',colorbar=False)
plt.title('Confusion Matrix Visualised');
plt.show()
```

```
TP = 5, TN=4, FP=3, FN=3
Accuracy = (TP+TN)/Total = 0.6 = 0.60
Precision = TP/(TP+FP) = 5/(5+3) = 0.62
Recall = TP/(TP+FN) = 5/(5+3) = 0.62
Confusion matrix:
[[4 3]
 [3 5]]
```



```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,f1_score,accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np
```

```
x_bal,y_bal=make_classification(n_samples=1000,weights=[0.5,0.5],random_state=1)
x_imb,y_imb=make_classification(n_samples=1000,weights=[0.99,0.01],random_state=1)
```

```
def evaluate(x,y,label):
    x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```

model=LogisticRegression(max_iter=1000).fit(x_train,y_train)
y_pred=model.predict(x_test)
acc=accuracy_score(y_test,y_pred)
f1=f1_score(y_test,y_pred,zero_division=0)
print(f'\n=== {label} ===')
print(f'Accuracy : {acc:.2%} <- {'MISLEADING!' if f1<0.5 else 'OK'}')
print(f'F1 Score : {f1:.2%} <- Real Performance')
print(classification_report(y_test,y_pred,zero_division=0))

```

```

evaluate(x_bal,y_bal,'balanced data')
evaluate(x_imb,y_imb,'imbalanced data')

```

```

=== balanced data ===
Accuracy : 80.50% <- OK
F1 Score : 80.00% <- Real Performance

```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```

=== imbalanced data ===
Accuracy : 80.50% <- OK
F1 Score : 80.00% <- Real Performance

```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```

x_train,x_test,y_train,y_test=train_test_split(x_imb,y_imb,test_size=0.2,random_state=42)
model_fixed = LogisticRegression(class_weight='balanced',max_iter=1000)
model_fixed.fit(x_train,y_train)
y_pred_fixed=model_fixed.predict(x_test)
print('\n=== After class_weight=balanced ===')
print(classification_report(y_test,y_pred_fixed,zero_division=0))

```

```

=== After class_weight=balanced ===

```

	precision	recall	f1-score	support
0	0.84	0.80	0.82	106
1	0.79	0.83	0.81	94
accuracy			0.81	200
macro avg	0.81	0.82	0.81	200
weighted avg	0.82	0.81	0.82	200

```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve,roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

```

```
x,y=make_classification(n_samples=800, n_features=10,random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

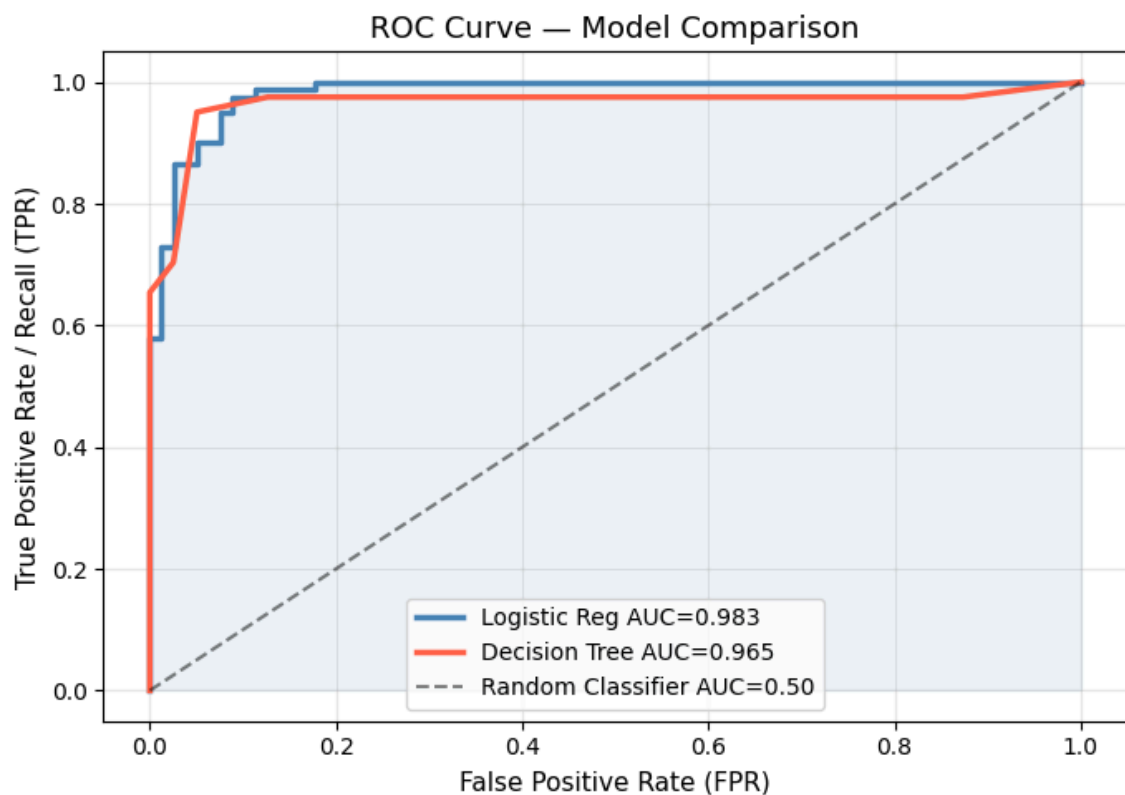
```
scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)
```

```
lr=LogisticRegression().fit(x_train,y_train)
probs_lr=lr.predict_proba(x_test)[:,-1]
```

```
dt=DecisionTreeClassifier(max_depth=4).fit(x_train,y_train)
probs_dt=dt.predict_proba(x_test)[:,-1]
```

```
fpr_lr, tpr_lr, _=roc_curve(y_test,probs_lr)
fpr_dt, tpr_dt, _=roc_curve(y_test,probs_dt)
auc_lr = roc_auc_score(y_test, probs_lr)
auc_dt = roc_auc_score(y_test, probs_dt)
```

```
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=2.5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve — Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



```
print(f'Logistic Regression AUC: {auc_lr:.4f}')
print(f'Decision Tree AUC: {auc_dt:.4f}')
print('Higher AUC = better model at separating classes')
```

```
Logistic Regression AUC: 0.9833
Decision Tree AUC: 0.9647
Higher AUC = better model at separating classes
```

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
ConfusionMatrixDisplay, roc_auc_score, roc_curve)
```

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
ConfusionMatrixDisplay, roc_auc_score, roc_curve)

# Download the dataset if not already present
!test -f diabetes.csv || wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.c

df = pd.read_csv('/content/diabetes.csv') # https://www.kaggle.com/datasets/uciml/pima-indians-di
print(df.shape) # (768, 9)
print(df['Outcome'].value_counts()) # 500 healthy, 268 diabetic
print(df.describe().round(2)) # stats: mean/std/min/max
# Distribution of each feature vs outcome
df.hist(bins=25, figsize=(14,10), color='steelblue', edgecolor='white')
plt.suptitle('Feature Distributions', fontsize=14); plt.tight_layout(); plt.show()
```

(768, 9)

Outcome

0 500

1 268

Name: count, dtype: int64

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
count	768.00	768.00	768.00	768.00	768.00	768.00	
mean	3.85	120.89	69.11	20.54	79.80	31.99	
std	3.37	31.97	19.36	15.95	115.24	7.88	
min	0.00	0.00	0.00	0.00	0.00	0.00	
25%	1.00	99.00	62.00	0.00	0.00	27.30	
50%	3.00	117.00	72.00	23.00	30.50	32.00	
75%	6.00	140.25	80.00	32.00	127.25	36.60	
max	17.00	199.00	122.00	99.00	846.00	67.10	

	DiabetesPedigreeFunction	Age	Outcome
count	768.00	768.00	768.00
mean	0.47	33.24	0.35
std	0.33	11.76	0.48